

Borg v2 — Hardened System Design

Consolidated revision of “Overcoming Borg’s Architectural Limitations”. This document keeps the essay’s five-limitation analysis and staged philosophy, corrects the defects in its code, fills the missing robustness layers, and compresses three-alternative solution sprawl into one canonical path per concern plus explicit upgrade triggers.

0. Executive Summary

The v1 essay correctly identifies Borg’s five growth constraints — GIL contention, no multi-host coordination, the TinyLlama quality ceiling, binary-options specificity, and missing input validation — and its “measure before you scale” philosophy is right. However, analysis of the essay as an engineering specification surfaces three problems:

- 1. **It is not safe to implement as written.** At least six of its code sketches contain defects that would break the system in production — including a job queue whose workers can never return results, a distributed lock that deadlocks permanently after the first expiry, and a work-partitioning scheme that is non-deterministic across processes.
- 2. **It is incomplete.** Robustness is not only concurrency and validation. The essay never addresses graceful shutdown, retry taxonomy, idempotency, the dual-write problem, observability, circuit breakers, authentication, configuration validation, backpressure policy, forecast calibration, data retention, or testing.
- 3. **It over-builds too early and is partly out of date.** Redis, Consul, Kafka, and microservices are introduced hundreds of times below the load that justifies them — a PostgreSQL SKIP LOCKED queue alone sustains ~850 jobs/min against Borg’s ~2 jobs/min target ¹². Meanwhile the GIL analysis predates Python 3.14’s officially supported free-threaded build ¹⁷, and the model comparison predates the Qwen3 generation of sub-4B models ¹⁶.

This v2 specification replaces v1’s fifteen solution sketches with **six canonical mechanisms**: an asyncio core with one bounded process pool, a Postgres-native job queue, claim-based multi-instance coordination, a cascading LLM router with calibration, a uniform strategy/risk interface, and a cross-cutting reliability layer. Everything v1 proposed remains reachable — each deferred technology has an explicit, measurable adoption trigger.

What changed from v1 to v2

Area	v1 approach	v2 approach	Why
CPU parallelism	Custom pool + custom queue + microservices (3 sketches)	asyncio core + one <code>ProcessPoolExecutor</code> (spawn)	Same benefit, ~80% less code, no result-loss bug
Job queue	<code>multiprocessing.Queue</code> + in-memory result cache	PostgreSQL FOR UPDATE SKIP LOCKED table	Crash-safe, transactional, results actually visible ¹⁰
Multi-host	Table locks / Consul / event streams (3 sketches)	Instance registry + advisory xact locks + claim-based work	No expired-row deadlock, no static partitioning, no Redis

Table 1 – continued

Area	v1 approach	v2 approach	Why
Work partitioning	hash(symbol) % 3	Workers claim rows via SKIP LOCKED	Deterministic, self-balancing, survives instance churn
LLM quality	“Speculative decoding” hybrid / bigger models / fine-tune	Cascade router + structured output + calibration loop	Correct terminology, threshold derived from data, not guessed
Local models	TinyLlama → Phi-3-mini → Mistral-7B	Qwen3-1.7B → Qwen3-4B (then fine-tune)	Newer generation, Apache-2.0, thinking toggle, Ollama-ready ¹⁶
Validation	Pydantic + rate limit + versioning (mostly right)	Same, with v2-correct Pydantic and un-broken plumbing	Fixes decorator-order, empty-file, and deprecation defects ²¹
Robustness	Not covered	Retry taxonomy, idempotency, outbox, shutdown, metrics, auth, config validation	The “robust” part of the mandate
Scale-out tech	Redis/Consul/Kafka/ microservices at Prompt 4+	Deferred behind explicit load triggers	Less infrastructure = fewer failure modes at 4 GB

1. Analysis of the v1 Essay

1.1 What holds and is kept

The essay’s skeleton is sound and v2 preserves it: the five limitations are real and correctly prioritized (validation first, then concurrency, then quality, then scale, then generalization). Database-centric co-ordination is the right instinct for a system that already runs PostgreSQL — adding a second stateful service before the first is saturated is a classic over-engineering failure. Pydantic-first input validation, hybrid local/cloud inference, strategy composition, and schema versioning are all the right tools. The strongest idea in the essay is its gating philosophy — *no optimization without a measurement justifying it* — and v2 hardens that idea into explicit phase-exit gates (Section 3) so it cannot be skipped under enthusiasm.

1.2 Defects found in v1 code

Severity: **B** = breaks the feature outright, **H** = breaks under realistic failure/load, **M** = correctness or maintainability hazard.

#	Sev	v1 location	Defect	Consequence	v2 fix
1	B	<code>JobDispatcher.results</code>	Placeholder written by worker <i>processes</i> ; multiprocessing gives each process private memory	Main process never sees any result; every job polls to timeout	Results in the Postgres jobs table (§2.3)
2	B	<code>DistributedLock.acquire</code>	<code>INSERT ... ON CONFLICT DO NOTHING</code> on the PK, but expired rows are never deleted	After the first TTL expiry the row blocks all future acquires — permanent, silent deadlock	<code>pg_try_advisory_x</code> (§2.4)
3	B	Lock SQL	<code>INTERVAL '%d seconds' —</code> psychopg only binds <code>%s/%(name)s</code> ; <code>%d</code> is sent literally	invalid input syntax on every acquire	Advisory-lock function call; no interval math
4	B	Forecast partitioning	<code>hash(symbol)</code> is salted per process (<code>PYTHONHASHSEED</code>); <code>uuid.uuid4().int</code> regenerated per call; divisor hardcoded to 3	Instances compute <i>different</i> partitions → duplicate forecasts on some symbols, zero coverage on others	Claim-based work stealing, no static partition (§2.4)
5	H	Threads + Pool	Thread pools exist before <code>multiprocessing.Pool</code> forks	Fork-with-threads can deadlock at import/lock state	<code>spawn</code> context; pool created at startup, before threads (§2.2)

Table 2 – continued

#	Sev	v1 location	Defect	Consequence	v2 fix
6	H	Upload endpoint	<code>@rate_limit</code> applied under <code>@app.post</code> replaces the registered function with <code>wrap_per(request, *args); file.size</code> may be <code>None</code> ; temp file leaks when the size guard trips	FastAPI loses the <code>UploadFile</code> signature → broken route; disk fills with orphaned temp files	Rate limit as <code>Depends/</code> middleware; <code>finally</code> -guarded cleanup (§2.6)
7	H	<code>EnrichedForecastStrategy</code>	<code>analyze(market_data)</code> with <code>analyze(symbol)</code>	Liskov violation —coordinator passes a dict, strategy expects a str → runtime failure	Uniform <code>MarketContext</code> for all strategies (§2.7)
8	H	<code>_allocate_capital</code>	<code>abs(entry - stop_loss)</code> with <code>stop_loss=None</code> (binary strategy never sets one)	<code>TypeError</code> on the first unstopped trade	Position-size cap when no stop exists (§2.7)
9	H	<code>forecast_phase</code>	<code>asyncio.run()</code> called inside an already-running loop / once per forecast	<code>RuntimeError</code> , or event-loop churn per forecast	Single async brain loop; <code>await</code> the router (§2.5)
10	H	Consul <code>elect_leader</code>	Session never renewed (and <code>import time</code> missing)	Leadership silently lost after TTL; split-brain on next election	Consul deferred; if adopted, background renew loop is mandatory
11	H	Redis event stream	<code>XADD</code> unbounded; no consumer groups; restart replays from ID <code>"0"</code>	Memory growth forever; reprocessing storm after every crash	Outbox + relay, <code>MAXLEN ~</code> trimming, groups when needed (§2.8)

Table 2 – continued

#	Sev	v1 location	Defect	Consequence	v2 fix
12	H	Forecast/event writes	No unique constraint or idempotency key	Retries and multi-instance races double-insert the same candle’ s forecast	UNIQUE(symbol, ts, horizon) + ON CONFLICT DO NOTHING (§2.3)
13	M	Pydantic models	class Config deprecated in v2, removal slated for v3; confidence validator raises while its docstring says “cap at 95”	Deprecation warnings → future breakage; contradictory behavior ²¹	model_config = ConfigDict(...); clamp explicitly (§2.6)
14	M	ingest_candles_safe	row_num referenced after the loop; db is an undeclared global	NameError on an empty CSV; untestable function	Initialized counters; injected db (§2.6)
15	M	GIL root-cause	Claims blocking httpx calls hold the GIL; they release it while waiting on the socket (inference runs in Ollama’ s Go process anyway)	Optimization aimed at the wrong target	Profile first: real CPU hogs are Pydantic/JSON parsing and Python-side vector math (§2.2)
16	M	Embedding claim	“Latency drops ~50% with 2 workers” —two workers double <i>throughput</i> ; per-batch latency is unchanged (the batch loop is sequential)	Wrong SLO expectations	Latency vs throughput stated separately (§2.2)

1.3 Conceptual corrections

The GIL section is one year out of date. Since Python 3.14 (October 2025), the free-threaded (no-GIL) CPython build is officially supported under PEP 779 — no longer experimental — with single-thread overhead reduced from ~40% to roughly 5–10% and about 51% of the most-downloaded native

packages shipping free-threaded wheels ¹⁴. This does not change v2's recommendation for a 4 GB edge host: separate processes still buy memory isolation and crash containment, which threads never will. But "threads can't parallelize CPU work" is no longer a law of Python — it is a property of the default build. Free-threaded Python (3.14t) becomes a legitimate re-evaluation point at Phase C, once profiling data exists.

"Speculative decoding" is misnamed. The essay's hybrid is cascade (escalation) routing: a cheap model answers, an expensive model re-checks low-confidence answers. Speculative decoding is a different, token-level technique where a draft model proposes tokens that a target model verifies in parallel. The distinction matters because real speculative decoding requires draft/target logit compatibility and would not involve an API call per escalation; v2 uses the correct term throughout so future contributors don't implement the wrong thing.

Static partitioning is the wrong primitive for multi-instance work. v1 partitions symbols by hash, which couples correctness to every instance computing identical hashes and knowing the instance count. Claim-based work stealing (FOR UPDATE SKIP LOCKED) achieves the same "no two instances forecast the same candle" guarantee with zero coordination state: the row lock *is* the partition, it rebalances automatically when an instance dies, and a worker crash rolls the job back to pending for free ¹⁰.

The infrastructure threshold math doesn't support early Redis. v1 introduces Redis at Prompt 3–4. Published benchmarks put a Postgres SKIP LOCKED queue at ~850 jobs/min versus ~920 for Redis + BullMQ, with failure recovery handled by transaction rollback instead of bespoke stalled-job detection ¹². Borg's stated scale target is >100 forecasts/hour — roughly **1.7 jobs/min, i.e. 500× headroom** before Postgres becomes the bottleneck. v2 therefore deletes Redis from the critical path and keeps it strictly as a documented upgrade trigger.

1.4 Missing layers (the completeness gaps)

Robustness is a property of the whole lifecycle, and v1 specifies only the happy path. Fourteen gaps must be closed for the system to be considered complete:

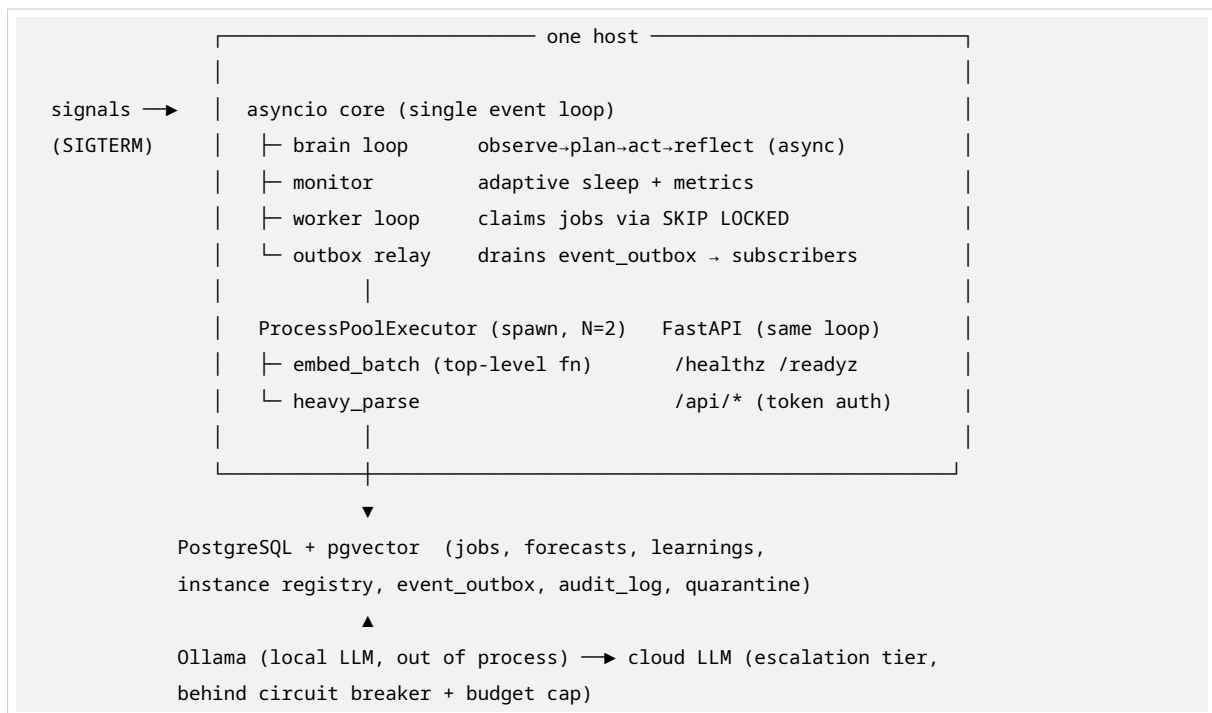
1. **Graceful shutdown** — nothing drains in-flight jobs or checkpoints state on SIGTERM.
2. **Retry taxonomy** — every failure path in v1 is `print(...)` and drop; transient vs permanent errors are indistinguishable.
3. **Idempotency** — no dedup keys on jobs, forecasts, or events; retries are unsafe.
4. **Outbox pattern** — v1 writes state to Postgres and publishes events to Redis as two separate operations (the dual-write problem: one can succeed without the other).
5. **Observability** — no metrics, no structured logs, no correlation IDs; the debugging complaint in v1's own cons list ("traces span processes") has no remedy.
6. **Circuit breakers** — the Ollama-down fallback is mentioned but never specified; a sick dependency must fail fast, not stack up callers.
7. **Authentication** — rate limiting is specified for an API that has no auth; unauthenticated rate limits only throttle honest users.
8. **Configuration validation** — a bad `.env` currently fails at first use, not at boot.
9. **Health/readiness endpoints** — required by any supervisor, watchdog, or orchestrator; absent.
10. **Backpressure policy** — `Queue(maxsize=100)` appears once, with no statement of what blocks, what sheds, and what alerts when full.
11. **Calibration loop** — the 65% escalation threshold is a guess and is never re-derived from resolved forecast outcomes.
12. **Data retention** — candles, learnings, events, and audit rows grow without bound on a 4 GB host.

13. **Testing strategy** — none: no property tests for validators, no integration tests, no replay harness.
14. **Migration management** — `schema.sql` is applied ad hoc; schema evolution needs versioned migrations from day one.

2. Hardened Architecture (v2)

2.1 Architecture at a glance

Single host, three concurrency domains, one database — this is the complete target for Phases A–C. Multi-instance is the same diagram replicated, with Postgres as the only shared component (§2.4).



Design rules that everything below obeys: **one event loop per process** (no `asyncio.run` inside the loop), **the database is the only cross-process/cross-instance shared state**, **every external effect is idempotent**, and **every deferred technology has a measurable trigger, not a date**.

2.2 Concurrency core: asyncio + one bounded process pool

v1's three sketches (pool, custom queue, microservices) collapse into one mechanism. The asyncio loop owns all I/O (database, Ollama HTTP, web). Genuinely CPU-bound work — embedding batch prep, large JSON/Pydantic parsing, vector math done in Python — goes to a `ProcessPoolExecutor` created with the `spawn` context **at startup, before any thread exists**, which removes v1's fork-with-threads deadlock class (defect #5). Worker functions must be **module-level** so they pickle cleanly under spawn.

```
# borg/executor.py
import asyncio, multiprocessing as mp
from concurrent.futures import ProcessPoolExecutor
from typing import Callable, TypeVar

T = TypeVar("T")
```

```

class CpuPool:
    """The single sanctioned way to run CPU-bound work off the event loop."""

    def __init__(self, workers: int = 2):
        ctx = mp.get_context("spawn")          # never fork a threaded process
        self._pool = ProcessPoolExecutor(max_workers=workers, mp_context=ctx)

    async def run(self, fn: Callable[..., T], *args) -> T:
        loop = asyncio.get_running_loop()
        return await loop.run_in_executor(self._pool, fn, *args)

    def shutdown(self):
        self._pool.shutdown(wait=False, cancel_futures=True)

# borg/tasks.py — worker functions live at module level (picklable under spawn)
def embed_batch(texts: list[str], model: str, base_url: str) -> list[list[float]]:
    import httpx                                # imported inside the worker
    with httpx.Client(base_url=base_url, timeout=30) as client:
        resp = client.post("/api/embed", json={"model": model, "input": texts})
        resp.raise_for_status()
    return [item["embedding"] for item in resp.json()["data"]]

```

Usage in the brain is one line — `vectors = await cpu_pool.run(embed_batch, texts, model, ollama_url)` — and the storage path never blocks: insert the learning row immediately with `embedding NULL`, submit the embedding as a job (§2.3), backfill on completion. Two honest expectation corrections against v1: **two workers double embedding throughput but do not halve single-batch latency** (defect #16), and most “GIL freezes” in the current codebase are actually blocking I/O — switching the brain to `httpx.AsyncClient` and an async DB driver resolves them before any process pool is needed (defect #15). Profile with `py-spy` for one hour before raising `workers` above 2 on a 4 GB host; each spawned interpreter costs ~50–100 MB.

Adaptive backpressure (replaces v1’s monitor sketch): the monitor loop reads `psutil.cpu_percent(interval=None)` each cycle; above 80% the brain lengthens its sleep and the worker loop stops claiming *low-priority* jobs; above 90% it sheds optional work (reflection, enrichment) entirely and emits a `borg_overload` metric. Backpressure is a policy table, not an afterthought:

Signal	Threshold	Action
CPU	>80% for 60 s	Brain sleep ×5; skip priority-0 jobs
CPU	>90% for 60 s	Shed optional work; alert
Queue depth	>500 pending	Reject new low-priority submissions (429 at API)
Memory	<300 MB free	Refuse new uploads; shed enrichment
Embedding NULL backlog	>1000 rows	Raise embed job priority; alert

Strategic option, not a plan: Python 3.14’s free-threaded build is officially supported with ~5–10% single-thread overhead and ~51% native-wheel coverage ³⁷. At Phase C, if profiling shows Python-side CPU as the top cost and Borg’s dependency set is wheel-clean, benchmark the brain loop under 3.14t

with threads instead of processes. Processes remain the default for their memory isolation; this is a re-evaluation point, not a migration commitment.

2.3 Job queue: PostgreSQL SKIP LOCKED (the only queue)

One table replaces v1's multiprocessing.Queue, its broken result cache, and the early Redis. FOR UPDATE SKIP LOCKED is the idiomatic Postgres worker-queue primitive: workers skip rows locked by peers, the row lock is the claim, and the pattern benchmarks at ~850 jobs/min — Borg needs ~2¹⁰¹².

```
CREATE TABLE jobs (  
  id          bigserial PRIMARY KEY,  
  kind        text NOT NULL,          -- 'embed' | 'forecast' | 'reflect' | ...  
  payload     jsonb NOT NULL,  
  idempotency_key text NOT NULL UNIQUE, -- deterministic: kind:symbol:ts:horizon  
  status      text NOT NULL DEFAULT 'pending'  
              CHECK (status IN ('pending', 'running', 'done', 'failed', 'dead')),  
  priority    int  NOT NULL DEFAULT 0,  
  attempts    int  NOT NULL DEFAULT 0,  
  max_attempts int  NOT NULL DEFAULT 3,  
  run_at      timestampz NOT NULL DEFAULT now(), -- backoff lives here  
  claimed_by  text,  
  claimed_at  timestampz,  
  finished_at timestampz,  
  result      jsonb,  
  error       text,  
  created_at  timestampz NOT NULL DEFAULT now()  
);  
-- partial index keeps dequeue fast and small  
CREATE INDEX jobs_dequeue_idx ON jobs (priority DESC, run_at)  
  WHERE status = 'pending';  
  
-- forecasts get the same idempotency treatment (defect #12)  
ALTER TABLE forecasts  
  ADD CONSTRAINT forecasts_idem UNIQUE (symbol, ts, horizon_minutes);
```

The claim is a single atomic statement (no separate SELECT-then-UPDATE race):

```
UPDATE jobs  
SET status = 'running', claimed_by = $1, claimed_at = now(), attempts = attempts + 1  
WHERE id = (  
  SELECT id FROM jobs  
  WHERE status = 'pending' AND run_at <= now()  
  ORDER BY priority DESC, run_at  
  LIMIT 1  
  FOR UPDATE SKIP LOCKED  
)  
RETURNING *;
```

The worker loop claims in a **short** transaction, executes the handler *outside* any transaction (LLM calls must not hold row locks), then finalizes in a second short transaction. Crash safety is explicit, not assumed:

- **Retry with backoff:** on failure, `status='pending'`, `run_at = now() + (2^attempts * interval '5 seconds')`, `error=$2` until `attempts = max_attempts`, then `status='dead'` and an alert.
- **Stale-claim reaper:** a monitor query resets running rows whose `claimed_at < now() - interval '5 minutes'` back to `pending`. This covers worker death mid-job (since the work happens outside the claim transaction, crash recovery cannot be left to rollback alone).
- **Idempotent submission:** producers `INSERT ... ON CONFLICT (idempotency_key) DO NOTHING RETURNING id;` a duplicate submission returns the existing job instead of doing the work twice.
- **Bounded results:** `finished_at` plus a nightly `DELETE FROM jobs WHERE status IN ('done', 'dead') AND finished_at < now() - interval '7 days'` keeps the table small on the 4 GB host (\$2.8 retention).

2.4 Multi-instance coordination: registry + advisory locks + claiming

v1 offered table locks (broken, defects #2-3), Consul (premature), and event streams (premature). v2 needs no new infrastructure — the shared Postgres already provides everything, in three pieces.

1. Instance registry with heartbeat replaces static instance counts:

```
CREATE TABLE instances (
  instance_id text PRIMARY KEY,
  role        text NOT NULL DEFAULT 'worker',
  started_at  timestampz NOT NULL DEFAULT now(),
  heartbeat_at timestampz NOT NULL DEFAULT now()
);
```

Each instance upserts its heartbeat every 10 s; peers with `heartbeat_at > now() - interval '30 seconds'` are alive. The dashboard reads this table — “3× tasks, unclear which is real” (v1’s scenario) disappears because every forecast row already carries `created_by`.

2. Singleton phases use transaction-scoped advisory locks. Observe and reflect must run on exactly one instance per cycle. `pg_try_advisory_xact_lock` auto-releases on commit *and on disconnect* — no TTL tables, no expired-row deadlock, no renewal loop:

```
def observe_phase(db):
    with db.transaction() as tx:
        got = tx.scalar("SELECT pg_try_advisory_xact_lock(hashtext('borg:observe'))")
        if not got:
            log.info("observe held by peer; skipping cycle")
            return
        candles = fetch_market_data()
        tx.execute(INSERT_CANDLES_SQL, candles) # ON CONFLICT (symbol, ts) DO NOTHING
```

3. Forecasting uses claim-based work stealing (§1.3): each instance’s worker loop claims pending forecast jobs from the §2.3 table. There is no partition to compute, so there is nothing to get wrong when instances join, leave, or die — the queue rebalances by construction. This fixes defect #4 by deletion.

Escalation triggers (explicit, measurable): move the queue to Redis Streams only when sustained depth exceeds ~500 jobs/min or p99 claim latency exceeds 50 ms ¹²; add Consul/etcd only when coordination extends beyond one Postgres (multi-region writes) or instances exceed ~20 and advisory-lock waits, observable in `pg_stat_activity`, consistently exceed `lock_timeout` ¹²; adopt Kafka only for event *sourcing* requirements (replay/audit beyond the outbox’s 7-day window).

2.5 LLM layer: cascade router, structured output, calibration

Three changes against v1: correct terminology (cascade, not “speculative decoding”), structured output instead of regex-over-prose, and a threshold that comes from data.

Model tier update (2026). TinyLlama is two generations behind. The Qwen3 dense line — 0.6B / 1.7B / 4B, Apache-2.0, thinking/non-thinking modes, first-class Ollama support — is the current default for this footprint, with even the 4B reported by its authors to rival Qwen2.5-72B-Instruct ¹⁶. Treat the table as configuration, not code, and re-benchmark on Borg’s own prompt set before adopting:

Tier	Model	Params	~RAM (Q4)	Role
Local default	Qwen3-1.7B	1.7B	~1.4 GB	All routine forecasts
Local quality	Qwen3-4B	4B	~2.6 GB	8 GB hosts; low-confidence second pass
Cloud escalation	current mini-class API (OpenAI/ Anthropic/Google — pick at deploy time)	—	—	Only escalated calls, budget-capped

Cascade router. The escalation rule has two triggers — low confidence *or* invalid/unparseable structured output — and sits behind a circuit breaker and a daily cloud budget cap so a sick API or a runaway loop fails fast and cheap:

```
# borg/llm_router.py
class LlmRouter:
    def __init__(self, local: LocalLlm, cloud: CloudLlm | None,
                 threshold: float, breaker: CircuitBreaker, budget: DailyBudget):
        self.local, self.cloud = local, cloud
        self.threshold, self.breaker, self.budget = threshold, breaker, budget

    async def forecast(self, ctx: MarketContext) -> Forecast:
        # Ollama structured output: pass the Pydantic JSON Schema as `format`
        raw = await self.local.generate_json(prompt(ctx), ForecastOut.json_schema())
        local_fc = ForecastOut.model_validate(raw)          # raises -> counts as invalid
        if local_fc.confidence >= self.threshold:
            return self._record(local_fc, model="local")

        if self.cloud and self.breaker.allow() and self.budget.allow():
            try:
                raw = await self.cloud.generate_json(review_prompt(ctx, local_fc),
                                                       ForecastOut.json_schema())
                cloud_fc = ForecastOut.model_validate(raw)
                self.breaker.ok()
                return self._record(cloud_fc, model="cloud",
                                    escalated_from=local_fc.confidence)
            except (ExternalServiceError, ValidationError) as e:
                self.breaker.fail(e)          # 3 consecutive failures -> open 30 s
        return self._record(local_fc, model="local", note="escalation unavailable")
```

The breaker is a 15-line state machine (closed → open after N consecutive failures → half-open probe after cooldown); it replaces v1's hand-waved “fallback if OpenAI is down.” `ForecastOut` is the same Pydantic model used for validation *and* as the Ollama `format` schema, so “model returned prose” becomes structurally impossible rather than regex-patched (v1's `_parse_json` disappears).

Calibration loop (new, closes gap #11). Every forecast row stores `model_used`, `confidence`, and later `outcome`. A nightly job computes Brier score and a reliability curve per model per symbol bucket, and writes the escalation threshold that maximizes expected accuracy per dollar into the config table. The threshold is thus a *measured* quantity reviewed weekly — v1's hardcoded 65 was a placeholder that would otherwise fossilize.

Fine-tuning path (kept, corrected). v1's sketch was directionally right but premature and its deployment step was speculative. The durable path: accumulate 1,000 resolved forecasts with outcomes → distill cloud-tier reasoning into (prompt, ideal-analysis) pairs → LoRA with PEFT/unsloth on a rented GPU → **merge into the base weights, convert to GGUF, and ollama create borg-qwen FROM ./merged.gguf** — do not depend on Modelfile ADAPTER support, which is not a reliable runtime path. Gate: fine-tune only if the calibration loop shows the local model's Brier score plateauing *and* the cascade's cloud spend exceeds the GPU rental cost over a quarter.

2.6 Validation & ingestion (corrected and completed)

v1's instinct was right and its plumbing was broken (defects #6, #13, #14). The corrected layer has four parts.

1. Schemas — current Pydantic v2 idiom. `model_config = ConfigDict(...)`, validators that do what their docstrings say:

```
# borg/schemas.py
from datetime import datetime
from pydantic import BaseModel, ConfigDict, Field, model_validator

class CandleIn(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)

    symbol: str = Field(min_length=2, max_length=12, pattern=r"^[A-Z0-9.-]+")
    ts: datetime
    open: float = Field(gt=0, lt=1e9)
    high: float = Field(gt=0, lt=1e9)
    low: float = Field(gt=0, lt=1e9)
    close: float = Field(gt=0, lt=1e9)
    volume: float = Field(ge=0, le=1e15)

    @model_validator(mode="after")
    def ohlc_consistent(self):
        if not (self.low <= min(self.open, self.close)
                and max(self.open, self.close) <= self.high):
            raise ValueError("open/close outside [low, high]")
        return self

class ForecastOut(BaseModel):
    model_config = ConfigDict(extra="forbid")
    direction: str = Field(pattern="^(up|down|flat)$")
```

```

confidence: float = Field(ge=0, le=100)
rationale: str = Field(default="", max_length=1000)

@model_validator(mode="after")
def clamp_overconfidence(self):
    object.__setattr__(self, "confidence", min(self.confidence, 95.0))
    return self

```

2. Ingest pipeline — never crashes, never trusts. Every rejected row goes to a quarantine table with the raw payload and the error, so bad data is inspectable instead of silently dropped; every batch writes one audit row; counters are initialized so empty files can't NameError (defect #14); db is injected, not global.

```

# borg/ingest.py
import csv
from pydantic import ValidationError
from borg.schemas import CandleIn

INSERT_CANDLE = """
    INSERT INTO market_candles (symbol, ts, open, high, low, close, volume)
    VALUES (%(symbol)s, %(ts)s, %(open)s, %(high)s, %(low)s, %(close)s, %(volume)s)
    ON CONFLICT (symbol, ts) DO NOTHING
"""

def ingest_csv(path: str, db, actor: str = "cli") -> dict:
    inserted = skipped = 0
    for n, row in enumerate(csv.DictReader(open(path, newline="")), start=2):
        try:
            candle = CandleIn.model_validate(row)
            inserted += db.execute(INSERT_CANDLE, candle.model_dump()).rowcount
        except (ValidationError, ValueError, KeyError) as e:
            skipped += 1
            db.execute(
                "INSERT INTO quarantine (source, row_num, raw, error) VALUES (%s,%s,%s,%s)",
                (path, n, row, str(e)[:500]),
            )
    db.execute(
        "INSERT INTO audit_log (actor, action, detail) VALUES (%s, %s, %s)",
        (actor, "candle_ingest", {"file": path, "inserted": inserted, "skipped": skipped}),
    )
    return {"inserted": inserted, "skipped": skipped, "rows": inserted + skipped}

```

Semantic checks beyond schema (v1's Scenario 5) live in one SQL guard on insert — reject ts more than 5 minutes in the future, and flag (not reject) volume=0 or >10×-spread rows into quarantine for review — because “impossible” and “unusual” deserve different handling.

3. Upload endpoint — rate limiting that doesn't break FastAPI. v1's decorator replaced the route's signature (defect #6). Use a dependency, which FastAPI introspects separately from the handler:

```

# borg/web/deps.py
from fastapi import Depends, HTTPException, Request, UploadFile

```

```

async def rate_limit(request: Request, bucket: str, limit: int, window_s: int):
    key = f"rl:{bucket}:{request.client.host}"
    n = await request.app.state.limiter.incr(key, window_s) # DB or in-memory
    if n > limit:
        raise HTTPException(429, f"rate limit: {limit}/{window_s}s",
                             headers={"Retry-After": str(window_s)})

MAX_UPLOAD = 100 * 1024 * 1024

@app.post("/api/candles/upload",
          dependencies=[Depends(verify_token), # auth first ($2.8)
                        Depends(lambda r: rate_limit(r, "upload", 10, 3600))])
async def upload_candles(file: UploadFile):
    tmp_path = None
    try:
        tmp_path, size = await spool_to_disk(file, limit=MAX_UPLOAD) # chunked; 413 if over
        return await asyncio.to_thread(ingest_csv, tmp_path, db, actor="upload")
    finally:
        if tmp_path:
            os.unlink(maybe(tmp_path)) # no leak on the error path either

```

4. Schema versioning (kept, slimmed). v1's per-version model classes are fine but heavy for this scale. v2 keeps one internal model (CandleIn) plus a small upcaster registry keyed on an explicit **version** field in the payload — defaulting to 1 — so old producers keep working and new fields ride along as optional. The stored `schema_version` column for lineage is kept from v1.

2.7 Strategy framework: one interface, one risk chokepoint

v1's framework was the right idea with a broken contract (defects #7, #8). The corrected contract: **every strategy has the same signature**, strategies *propose* and the **RiskManager disposes** — it is the single chokepoint through which every order intent passes, so risk limits cannot be bypassed by adding a strategy.

```

# borg/strategies/base.py
from dataclasses import dataclass
from typing import Literal, Protocol

@dataclass(frozen=True)
class MarketContext:
    symbol: str
    candle: CandleIn
    features: dict # indicators, sentiment, macro — same object for all strategies

@dataclass(frozen=True)
class OrderIntent:
    symbol: str
    side: Literal["buy", "sell"]
    qty: float
    limit_price: float | None = None
    stop_loss: float | None = None # may be None; RiskManager handles it
    take_profit: float | None = None

```

```

strategy: str = ""
rationale: str = ""

class Strategy(Protocol):
    name: str
    def analyze(self, ctx: MarketContext) -> list[OrderIntent]: ...
    def risk_limits(self) -> dict: ...      # max_position_usd, max_drawdown_pct, max_leverage

# borg/risk.py
class RiskManager:
    def __init__(self, per_trade_risk_pct=0.02, daily_loss_halt_pct=0.05):
        self.per_trade_risk_pct, self.daily_loss_halt_pct = per_trade_risk_pct, daily_loss_halt_pct

    def approve(self, intents: list[OrderIntent], portfolio: Portfolio) -> list[OrderIntent]:
        if portfolio.daily_pnl_pct <= -self.daily_loss_halt_pct:
            return []                                # kill-switch: flat for the day
        out = []
        for i in intents:
            limits = min_risk_limits(i.strategy)      # most conservative of stack
            budget = portfolio.equity * self.per_trade_risk_pct
            qty = size_for(i, budget, limits)          # works when stop_loss is None:
            if qty > 0:                                # falls back to position-USD cap
                out.append(replace(i, qty=qty))
        return enforce_aggregate_exposure(out, portfolio)

```

Consensus (v1’s ConsensusStrategy) survives as one strategy among others — it takes the union of sub-strategy intents and emits only directions with 2 agreeing votes, weighted by each sub-strategy’s trailing 30-day hit rate from the calibration tables rather than flat votes. New strategies register in `strategies.yaml` and are exercised through the same replay harness as production (§2.9), which keeps backtest/live parity: the replay feeds historical `MarketContext` objects through the identical `analyze` code path.

2.8 Cross-cutting reliability layer (the missing half)

This section is entirely new relative to v1 and is what makes the system *robust* rather than merely *fast*.

Error taxonomy and retry policy. Three error classes, three policies — nothing is retried blindly and nothing is dropped silently:

Class	Examples	Policy
TransientError	DB connection reset, Ollama 503, rate-limit 429	Retry with exp backoff (base 5 s, ×2, +full jitter), max 3, then dead-letter
ExternalServiceError	Cloud LLM down, broker API 5xx	Circuit breaker (3 consecutive failures → open 30 s → half-open probe); degrade to local tier
PermanentError	Validation failure, schema mismatch, unknown job kind	No retry; quarantine/dead-letter + alert + audit row

Idempotency everywhere. Jobs (`idempotency_key`, §2.3), forecasts (`UNIQUE(symbol, ts, hori-`

zon_minutes)), candles (UNIQUE(symbol, ts)), and outbox events (UNIQUE(event_key)) all dedupe at the database, so every retry and every multi-instance race is safe by construction. Handlers must be written to be re-runnable; the reaper (§2.3) guarantees at-least-once, the constraints enforce effectively-once.

Outbox, not dual-write. State change and its event are inserted in the *same transaction* into event_outbox; a relay in the asyncio core drains the outbox to subscribers (local handlers now, Redis Streams later behind the §2.4 trigger) and marks rows published. If the process dies between commit and publish, the relay replays on restart — events can be redelivered but never lost, and subscribers must be idempotent (they are, by the constraints above).

Graceful shutdown. SIGTERM → set draining flag → API returns 503 on new writes, worker loops stop claiming, brain finishes its current cycle → wait (max 25 s) for in-flight jobs to finalize or return them to pending → flush outbox → close pool (cancel_futures=True) → close DB. systemd gets TimeoutStopSec=30. Unclaimed running rows are recovered by the reaper regardless, so a hard kill is recoverable, just slower.

Health, readiness, watchdog. /healthz (process up) and /readyz (DB reachable, Ollama reachable or breaker open-and-known, queue depth below shed threshold, heartbeat fresh). A watchdog cron restarts the service when /readyz fails 3 consecutive checks — covering the “system feels unresponsive” symptom v1 diagnosed but only treated inside the GIL section.

Observability. Structured JSON logs with trace_id propagated from API request → job claim → handler → LLM call (this answers v1’s own “traces span processes” caveat). Prometheus metrics, minimum set: borg_forecast_latency_seconds (histogram, by model), borg_jobs_claimed_total / borg_jobs_dead_total (by kind), borg_queue_depth, borg_escalations_total, borg_breaker_open (gauge), borg_ingest_rows_total{status}, borg_daily_loss_pct. Alerts: queue depth >500 for 10 min, dead jobs >0, breaker open >5 min, daily loss halt engaged.

Configuration as code. All settings via pydantic-settings, validated at boot (fail fast on a bad .env, gap #8): DSN, Ollama URL, model names, thresholds, budgets, risk limits. Two sources only — env vars and one YAML — with env winning; thresholds written by the calibration loop live in a config_overrides DB table and are re-read each cycle.

Security baseline (gap #7). Bearer-token auth on every /api/* route (verify_token above); tokens in env, hashed in DB if multiple users appear; upload quarantine as the only path for external data; parameterized SQL everywhere (already true); --max-requests jitter on uvicorn workers to blunt memory-leak tail risk.

Retention (gap #12). Nightly job, one table: candles >90 days → downsample to 5-min then drop raw; jobs/outbox >7 days → delete; audit >180 days → archive to CSV in /borg/output; learnings never auto-deleted but re-embedded only on demand.

2.9 Testing and calibration gates

v1 specified no tests; v2 makes four suites phase gates, not aspirations. **Property tests** (Hypothesis) fuzz CandleIn/ForecastOut and the OHLC validator with generated data, including the five failure scenarios from v1 §5 as regression cases. **Integration tests** (testcontainers-postgres) cover the claim/finalize/reaper cycle under a killed worker, the advisory-lock skip path, and idempotent resubmission. **Replay harness** replays recorded MarketContext history through any strategy and reports hit rate, Sharpe, and max drawdown — required before a strategy may set live: true in config. **Calibration report** (weekly, §2.5) is itself tested: Brier computation is pinned against a hand-checked fixture. CI runs the first three on every change; the fourth is generated nightly and archived with the audit trail.

3. Consolidated Roadmap

Phases replace v1's "Prompt 1–5" framing; each has an **entry condition** and a measurable **exit gate**, so scale work cannot start on vibes. The mapping to v1 prompts is noted for continuity.

Phase (= v1)	Scope	Entry	Exit gate (measured)
A (P2)	Async brain loop, Pydantic validation + quarantine + audit, monitor + adaptive backpressure, boot-time config validation, health endpoints	—	Fuzz suite green; 0 unvalidated writes in 7 days; forecast p95 <500 ms with local model; / readyz wired to watchdog
B (P3)	CpuPool, SKIP LOCKED job queue + reaper + idempotency, outbox relay, graceful shutdown, Prometheus metrics	CPU >60% at Phase-A load, or queue depth growing across 3 consecutive days	p95 claim latency <20 ms; 0 lost jobs in a kill-9 soak test; CPU <60% at 2× load; queue depth flat over 7 days
C (P3–4)	Model upgrade (Qwen3-1.7B/4B), cascade router + breaker + budget, calibration loop, optional 3.14t benchmark	200 resolved forecasts with outcomes	Local Brier score improves 10% vs TinyLlama baseline; escalation 15% of calls; cloud spend within budget; threshold now data-derived
D (P4)	Instance registry + heartbeats, advisory-lock singletons, claim-based forecasting at 2–3 instances	A second host actually exists	Two-node 72 h soak: 0 duplicate forecasts (unique constraint holds), failover of observe phase <60 s after instance kill
E (P5+)	Strategy framework + RiskManager, consensus, data enrichment, fine-tune evaluation	Binary strategy 55% hit rate over 500 live forecasts	Replay Sharpe and drawdown within RiskManager limits on 6 months of history; paper-trade 30 days before any live multi-strategy order

Deferred with triggers (do not build early): Redis queue (>500 jobs/min sustained or claim p99 >50 ms ¹²), Consul/etcd (>20 instances or multi-region writes), Kafka (event-sourcing requirements beyond 7-day outbox), microservices (sustained >1,000 jobs/min or independent deploy cadence per component), free-threaded Python migration (Phase-C benchmark proves it on Borg's own workload ³).

Service-level objectives

SLO	Target	Alert
Forecast latency, local	p95 <500 ms	p95 >1 s for 15 min
Escalated forecast latency	p95 <3 s	p95 >5 s for 15 min
Job loss	0 (dead-lettered lost)	any unaccounted running row after reaper pass
Ingest rejection rate	<1% of rows	>5% in any batch
Queue depth	<100 steady-state	>500 for 10 min
Daily loss halt	engages at -5%	engaged (informational) + failed to engage (critical)
Uptime (single host)	99% monthly	watchdog restarts >3/day

Decision rules (compact)

- 1. **CPU-bound?** Fix blocking I/O first (async everywhere); then CpuPool at 2 workers; raise only with py-spy evidence.
- 2. **Need a queue?** Postgres SKIP LOCKED until ~500 jobs/min sustained — that is ~300× current target load [12](#).
- 3. **Second host?** Registry + advisory locks + claiming; nothing else until the §2.4 triggers fire.
- 4. **Forecast quality?** Measure Brier first. Local upgrade (Qwen3-4B) before cloud cascade; cascade before fine-tuning; fine-tune only on a measured plateau with the data to feed it.
- 5. **New strategy?** Must pass replay + 30 days paper before the RiskManager will size a live intent.
- 6. **New dependency (Redis/Consul/Kafka/service)?** Must cite the triggered threshold in the change request; “might need it someday” is not a trigger.

This document is an engineering design for a software system that interacts with financial markets. It is provided for general informational purposes only and does not constitute investment, legal, or other professional advice; any live trading remains entirely at the operator’s own risk.